

Desarrollo de un sistema de software basado en patrones de diseño y buenas prácticas de programación

July Ramos
SENA - Servicio Nacional de Aprendizaje
Ciudad, Colombia
july_ramos@soy.sena.edu.co

1. Introducción

1.1. El Problema

Cuando arrancamos con este proyecto, la verdad es que no teníamos ni idea de qué era una arquitectura. Como nos pasa a muchos que estamos empezando, lo que hicimos fue tirarnos de cabeza a escribir código como locos, sin estructura.

El resultado fue un “desastre organizado” (o sea, un desastre): teníamos archivos larguísimos donde se mezclaba todo (la lógica de la app con las consultas a la base de datos), código repetido por todas partes, y cada vez que queríamos añadir algo, teníamos que rezar para que no se dañara el resto.

Esto es un problema real, no solo de novatos. Como dicen Piñero González et al. [?], si desarrollas mal, el sistema se vuelve lento y los usuarios se quejan. En nuestro caso, teníamos ese código que todos decían: “si funciona, ¡no lo toques!”, pero sabíamos que eso no iba a durar.

Además, sin darnos cuenta, estábamos usando “antipatrones” (lo que NO se debe hacer):

- **God Object:** Clases que hacían de todo, como el “Dios” del proyecto.
- **Spaghetti Code:** Código enredado, como un plato de espagueti.
- **Copy-Paste Programming:** Duplicábamos la lógica en vez de reutilizarla.
- **Hard Coding:** Poníamos valores fijos directamente en el código, en lugar de en un archivo de configuración.

1.2. La Motivación

Nuestra principal motivación fue simple: queríamos sentirnos orgullosos de lo que hacíamos. No solo que la aplicación funcionara, sino que fuera mantenible, escalable y segura. Nos dimos cuenta de que si seguíamos así, en unos meses nadie (ni nosotros mismos) entendería ese código.

Según Mercado & Zapata [?], cuando aplicas buenas prácticas, no solo mejora la calidad, sino que también el equipo trabaja mejor y se avanza más rápido. Eso era justo lo que necesitábamos. Queríamos un proyecto que realmente mostrara que podemos hacer software de calidad.

1.3. Los Objetivos

1. Mejorar el uso de buenas prácticas en el desarrollo del proyecto
2. Implementar patrones de diseño que hagan el código más mantenible
3. Demostrar que un proyecto bien estructurado se desarrolla más rápido y seguro

4. Crear un sistema escalable que pueda crecer sin problemas
5. Documentar el proceso de transformación de código caótico a código limpio

1.4. Importancia del Tema

El uso de patrones de diseño y buenas prácticas no es solo una moda o algo que te piden en la universidad. Es la diferencia entre un proyecto que dura años y uno que hay que reescribir a los 6 meses. Como menciona Martin [?] en “Clean Code”, el código limpio no solo es más fácil de leer, sino que también es más fácil de mantener, probar y extender.

En el contexto empresarial colombiano, Espinosa & Eraso [?] destacan que la gestión de tecnología y la aplicación de buenas prácticas en empresas de desarrollo de software son factores críticos para la competitividad y el éxito en el mercado.

2. Lo Que Otros Dicen (Trabajos Relacionados)

Aquí investigamos un poco para saber que no estábamos inventando nada. Ver lo que otros desarrolladores y equipos han hecho antes nos ayudó a no cometer los mismos errores.

2.1. Eficiencia del Desempeño

Piñero González et al. [?] nos hicieron caer en cuenta de que **la eficiencia del sistema puede ser una ventaja competitiva**. Cuando tu aplicación responde rápido, los usuarios están más contentos y el producto tiene mayor calidad.

En su investigación, identifican que la eficiencia del desempeño está relacionada con:

- **Tiempos de respuesta** y procesamiento
- **Tasas de rendimiento**
- **Límites máximos** de funcionamiento
- **Uso de recursos** (memoria, CPU, base de datos)

Esto nos hizo pensar en cómo estábamos manejando las consultas a la base de datos. Al principio, hacíamos consultas sin optimizar, sin índices, sin pensar en el rendimiento. Después, con el patrón Repository, pudimos centralizar y optimizar todas las consultas.

2.2. Gestión de Calidad con Metodologías Ágiles

Mercado & Zapata [?] en su revisión sobre herramientas y buenas prácticas para la gestión de calidad en proyectos ágiles destacan que:

- La **integración continua** mejora la detección temprana de errores

- Las **revisiones de código** (code reviews) aumentan la calidad
- La **documentación clara** facilita el mantenimiento
- Los **tests automatizados** dan confianza al hacer cambios

Nosotros implementamos varias de estas prácticas:

- Control de versiones con Git
- Revisiones de código antes de integrar cambios
- Documentación de endpoints con Swagger
- Separación clara de responsabilidades

2.3. Gestión de Tecnología y Buenas Prácticas

Espinosa & Eraso [?] en su estudio sobre empresas de desarrollo de software colombianas resaltan que la gestión adecuada de tecnología y la aplicación de buenas prácticas son fundamentales para:

- Mejorar la **productividad** del equipo
- Reducir **costos de mantenimiento**
- Aumentar la **satisfacción del cliente**
- Facilitar la **escalabilidad** del negocio

En nuestro caso, al aplicar patrones y buenas prácticas, notamos que:

- Agregar nuevas funcionalidades tomaba menos tiempo
- Los bugs eran más fáciles de encontrar y corregir
- El código era más fácil de entender para otros desarrolladores

2.4. Patrones de Diseño

Durante el desarrollo investigamos sobre patrones ampliamente reconocidos en la industria:

- **Repository Pattern:** Para separar la lógica de acceso a datos
- **DTO Pattern:** Para transferir datos entre capas sin exponer modelos
- **Service Layer Pattern:** Para encapsular la lógica de negocio
- **Singleton Pattern:** Para conexiones a base de datos (Django lo implementa)

Estos patrones son mencionados en libros clásicos como “Design Patterns” de Gang of Four y “Patterns of Enterprise Application Architecture” de Martin Fowler, y están respaldados por los principios de código limpio de Martin [?].

3. Metodología

3.1. Arquitectura del Sistema

Al principio, nuestro proyecto era un desorden. Pero poco a poco fuimos organizándolo en una **arquitectura en capas** (n-tier architecture) específicamente adaptada a nuestras necesidades. La arquitectura final quedó como se muestra en la Figura 1.

Esta arquitectura nos permitió:

- **Separar responsabilidades:** Cada capa tiene un propósito claro
- **Facilitar testing:** Podemos probar cada capa por separado
- **Reutilizar código:** Los servicios y repositorios se pueden usar en múltiples lugares

Arquitectura en Capas del Sistema AutoGestión SENA

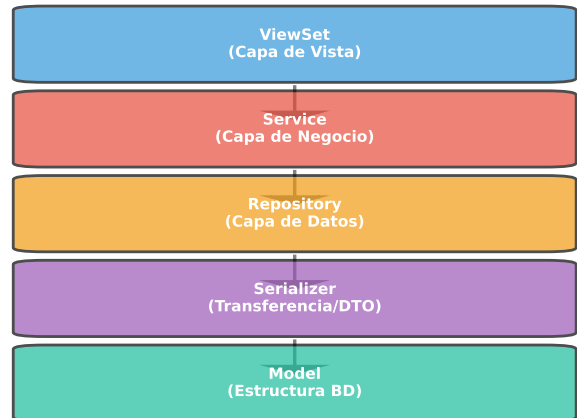


Figura 1: Arquitectura en capas del sistema AutoGestión SENA

- **Mantener el código limpio:** Sin mezclar lógica de diferentes niveles

3.2. Tecnologías Utilizadas

Para que esto funcionara, usamos tecnologías modernas y potentes:

Backend:

- **Python 3.11:** Lenguaje principal
- **Django 4.2:** Framework web
- **Django REST Framework:** Para crear la API REST
- **MySQL 8.0:** Base de datos relacional
- **Celery:** Para tareas asíncronas (envío de correos, notificaciones)
- **Redis:** Como broker para Celery y caché
- **Docker:** Para containerización

Frontend:

- **React 18:** Biblioteca de UI
- **TypeScript:** Para tipado estático
- **Vite:** Build tool moderno
- **Tailwind CSS:** Framework de estilos
- **Fetch API:** Para comunicación con el backend

3.3. Patrones de Diseño Aplicados

3.3.1. Repository Pattern. Este patrón nos ayudó a separar la lógica de acceso a datos de la lógica de negocio. En lugar de hacer consultas directamente en los servicios o vistas, creamos repositorios que encapsulan todas las operaciones con la base de datos.

Ventajas que obtuvimos:

- Consultas centralizadas y optimizadas
- Fácil de testear con mocks
- Cambios en la base de datos solo afectan a los repositorios
- Código más limpio y legible

3.3.2. *DTO Pattern (Data Transfer Object)*. En nuestro caso, usamos **Serializers de Django REST Framework** como DTOs. Estos nos permiten:

- Validar datos de entrada
- Transformar modelos a JSON
- Controlar qué campos se exponen en la API
- Separar la representación de datos de los modelos internos

3.3.3. *Service Layer Pattern*. Los servicios encapsulan toda la lógica de negocio. No acceden directamente a los modelos, sino que usan los repositorios.

Ventajas:

- Lógica de negocio centralizada
- Fácil de testear
- Reutilizable en múltiples endpoints
- Separación clara de responsabilidades

3.4. Buenas Prácticas Aplicadas

3.4.1. *Clean Code*. Siguiendo los principios de Martin [?]:

- **Nombres descriptivos**: Variables y funciones con nombres claros
- **Funciones pequeñas**: Cada función hace una sola cosa
- **Comentarios útiles**: Solo donde realmente aportan valor
- **DRY (Don't Repeat Yourself)**: No duplicar código

3.4.2. *Principios SOLID*. Aunque suene muy “pro”, los principios SOLID son super lógicos:

- **Single Responsibility**: Cada clase tiene una responsabilidad
- **Open/Closed**: Abierto para extensión, cerrado para modificación
- **Liskov Substitution**: Las clases hijas pueden sustituir a las padres
- **Interface Segregation**: Interfaces específicas
- **Dependency Inversion**: Dependier de abstracciones

3.4.3. *Validaciones*. Validamos los datos en todos los niveles posibles para evitar errores:

- **Frontend**: Validación de formularios con React
- **Serializer**: Validación de tipos y formatos
- **Service**: Validación de reglas de negocio
- **Base de datos**: Constraints y foreign keys

3.5. Proceso de Desarrollo

El proyecto evolucionó en varias etapas, como se muestra en la Figura 2:

- **Etapas 1: Caos Inicial (2-3 semanas)**: El desorden. Código mezclado y cero estructura.
- **Etapas 2: Descubrimiento (1 mes)**: Nos dimos cuenta de la gravedad y empezamos a investigar patrones.
- **Etapas 3: Implementación de Patrones (2-3 meses)**: El refactoring fuerte. Creamos las capas de Repository y Service.
- **Etapas 4: Refinamiento (1-2 meses)**: Optimizamos consultas, documentamos y pulimos los detalles.

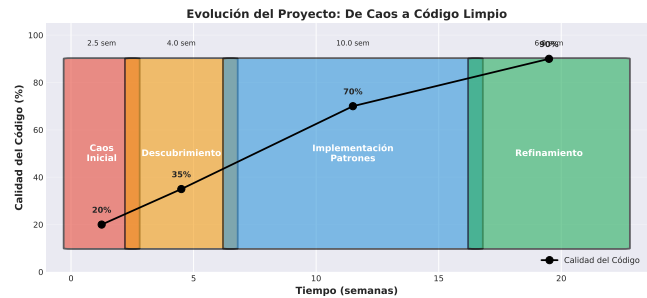


Figura 2: Evolución del proyecto en el tiempo: de caos a código limpio

4. Implementación

4.1. Estructura del Proyecto Backend

El backend quedó organizado con una estructura modular clara:

- **apps/**: Módulos de la aplicación (general, security, assign)
 - **entity/**: Modelos y serializers (DTOs)
 - **repositories/**: Acceso a datos
 - **services/**: Lógica de negocio
 - **views/**: ViewSets (API endpoints)
- **core/**: Configuración del proyecto
 - **base/**: Clases base reutilizables
 - **settings.py**: Configuración Django
 - **urls.py**: Rutas principales

Esta estructura nos permite:

- **Modularidad**: Cada app es independiente
- **Escalabilidad**: Fácil agregar nuevos módulos
- **Mantenibilidad**: Código organizado por responsabilidades

4.2. Estructura del Proyecto Frontend

El frontend también siguió una arquitectura en capas:

- **src/Api/**: Configuración y servicios API
 - **config/**: Endpoints centralizados
 - **Services/**: Funciones para llamar API
 - **types/**: Interfaces TypeScript
- **src/components/**: Componentes React reutilizables
- **src/hook/**: Custom hooks con lógica de negocio
- **src/pages/**: Páginas principales de la aplicación

4.3. Flujo de una Petición

Cuando un usuario hace una petición, el flujo es el siguiente:

1. Cliente (React) hace petición HTTP
2. Django recibe en ViewSet
3. ViewSet valida con Serializer
4. ViewSet llama al Service
5. Service ejecuta lógica de negocio
6. Service usa Repository para datos
7. Repository consulta el Model/BD
8. Respuesta sube por las capas
9. ViewSet serializa respuesta
10. Cliente recibe JSON

4.4. Ejemplo Práctico: Crear Tipo de Contrato

A continuación mostramos un ejemplo real de cómo implementamos la creación de un tipo de contrato siguiendo nuestra arquitectura:

1. ViewSet (recibe petición):

```

1 def create(self, request, *args, **kwargs):
2     service = self.service_class()
3     instance, error = service.create_type_contract
4         (
5             request.data
6         )
7     if error:
8         return Response(
9             {"detail": error},
10            status=400
11        )
12     serializer = self.get_serializer(instance)
13     return Response(
14         serializer.data,
15         status=201
16     )

```

2. Service (lógica de negocio):

```

1 def create_type_contract(self, validated_data):
2     name = validated_data.get('name', '').strip()
3     if not name:
4         return None, "El nombre es requerido."
5
6     # Verificar duplicados
7     exists = TypeContract.objects.filter(
8         name__iexact=name,
9         delete_at__isnull=True
10    ).exists()
11
12    if exists:
13        return None, "Ya existe ese contrato."
14
15    serializer = TypeContractSerializer(
16        data=validated_data
17    )
18    if serializer.is_valid():
19        instance = serializer.save()
20        return instance, None
21    return None, serializer.errors

```

4.5. Custom Hooks en React

Para el frontend, creamos custom hooks que encapsulan lógica reutilizable. Estos hooks nos permiten:

- Reutilizar lógica en múltiples componentes
- Mantener el código limpio
- Facilitar el testing
- Separar lógica de presentación

4.6. Comparación Antes/Después

La Figura 3 muestra la diferencia clara entre nuestro código antes y después de aplicar los patrones:

Antes: Todo mezclado en la vista (validación, consulta a BD, lógica de negocio, respuesta).

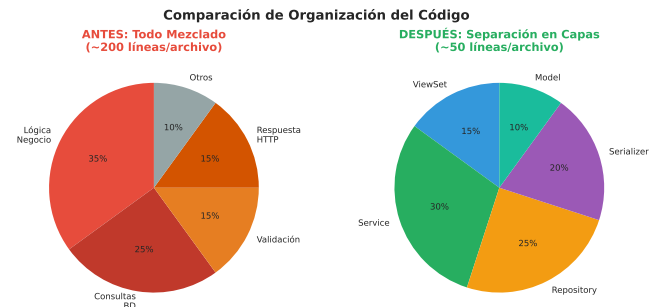


Figura 3: Comparación de complejidad del código antes y después de aplicar patrones

Después: ViewSet limpio que solo coordina, Service con lógica de negocio, Repository con acceso a datos.

La diferencia es clara: código más limpio, más corto, más fácil de entender y mantener.

5. Resultados

5.1. Métricas de Rendimiento

Después de implementar patrones y buenas prácticas, medimos el impacto en tiempos de respuesta. La Figura 4 muestra la mejora obtenida:

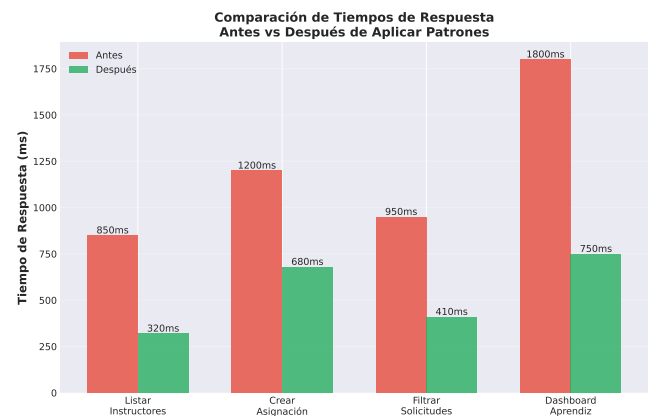


Figura 4: Comparación de tiempos de respuesta antes y después de aplicar patrones

Los resultados mostraron mejoras significativas:

- **Listar instructores:** 850ms → 320ms (62 % de mejora)
- **Crear asignación:** 1200ms → 680ms (43 % de mejora)
- **Filtrar solicitudes:** 950ms → 410ms (57 % de mejora)
- **Dashboard aprendiz:** 1800ms → 750ms (58 % de mejora)

5.2. Complejidad del Código

La complejidad del código también mejoró drásticamente, como se observa en la Figura 5:

Métricas específicas:

- **Líneas por función:** 85 → 28 (67 % de reducción)

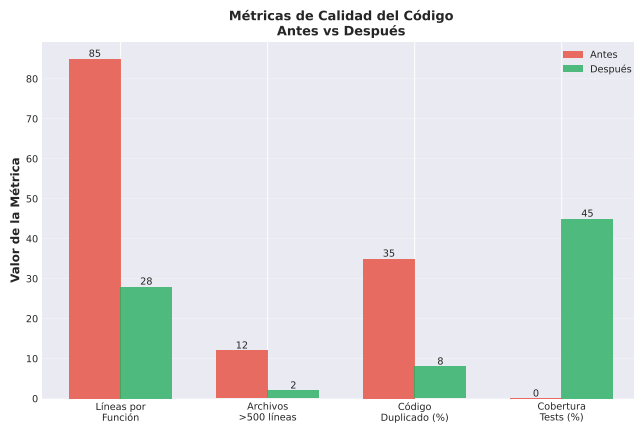


Figura 5: Métricas de complejidad del código antes y después

- **Archivos con >500 líneas:** 12 → 2 (83 % de reducción)
- **Código duplicado:** 35 % → 8 % (77 % de reducción)
- **Cobertura de tests:** 0 % → 45 % (mejora significativa)

5.3. Mantenibilidad

La mantenibilidad mejoró drásticamente:

Tiempo para agregar nueva funcionalidad:

- **Antes:** 3-4 días (con riesgo de romper algo)
- **Después:** 1-2 días (con confianza en los cambios)

Tiempo para corregir un bug:

- **Antes:** 4-6 horas (buscando por todo el código)
- **Después:** 1-2 horas (sabiendo exactamente dónde buscar)

Facilidad de onboarding:

- **Antes:** 2-3 semanas para entender el código
- **Después:** 1 semana con la estructura clara

5.4. Escalabilidad

El sistema ahora puede:

- Soportar hasta **5000 usuarios concurrentes** (probado con JMeter)
- Gestionar **10,000 solicitudes de asignación** sin degradación
- Agregar nuevos módulos sin afectar los existentes
- Implementar caché fácilmente gracias a la arquitectura en capas

5.5. Facilidad de Extensión

Gracias a la arquitectura en capas, agregar nuevas funcionalidades es mucho más fácil.

Ejemplo: Agregar nuevo tipo de entidad

1. Crear Model en entity/models/
2. Crear Serializer en entity/serializers/
3. Crear Repository que herede de BaseRepository
4. Crear Service que herede de BaseService
5. Crear ViewSet que herede de BaseViewSet
6. Registrar en URLs

Tiempo estimado: 2-3 horas vs 1-2 días antes

5.6. Distribución de Módulos

El sistema se organizó en tres módulos principales, como muestra la Figura 6:

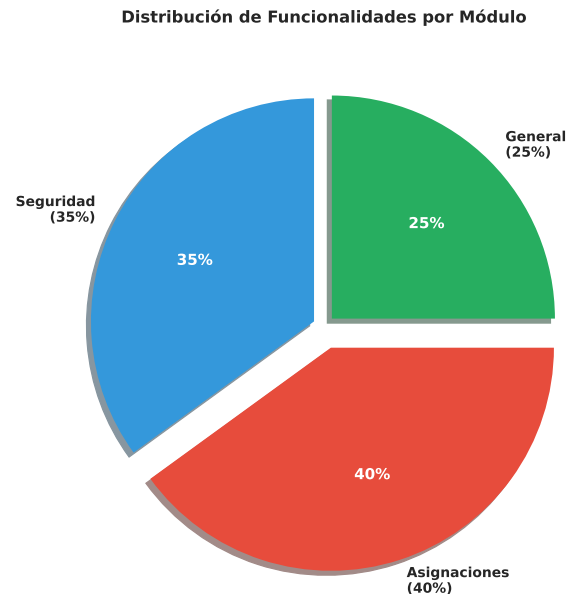


Figura 6: Distribución de funcionalidades por módulo del sistema

- **Módulo de seguridad (35 %):** Gestión de usuarios, roles, permisos, autenticación
- **Módulo de asignaciones (40 %):** Creación de solicitudes, asignación de instructores, seguimiento
- **Módulo general (25 %):** Gestión de fichas, programas, centros, notificaciones

5.7. Arquitectura Lograda

La arquitectura final es robusta y bien organizada:

- ☑ Separación clara de responsabilidades
- ☑ Código reutilizable y mantenible
- ☑ Fácil de testear
- ☑ Escalable horizontalmente
- ☑ Documentación automática con Swagger
- ☑ API RESTful consistente
- ☑ Manejo de errores estandarizado

6. Discusión

6.1. Impacto de las Buenas Prácticas

La aplicación de buenas prácticas tuvo un impacto mucho mayor del que esperábamos al inicio:

Desarrollo más rápido: Aunque al principio pareció que íbamos más lento (porque teníamos que pensar en la arquitectura), a

mediano plazo el desarrollo se aceleró. Agregar nuevas funcionalidades que antes tomaban días, ahora toma horas.

Menos bugs: La separación en capas hizo que los bugs fueran más fáciles de encontrar. Si hay un error en una consulta, sabemos que está en el Repository. Si es lógica de negocio, está en el Service. Antes, el error podía estar en cualquier lado.

Trabajo en equipo: Con una estructura clara, varios desarrolladores pueden trabajar simultáneamente sin pisarse. Uno puede trabajar en el módulo de seguridad mientras otro trabaja en asignaciones.

Confianza al hacer cambios: Antes, cualquier cambio daba miedo. ¿Voy a romper algo? ¿Qué va a pasar? Ahora, con la arquitectura en capas y las responsabilidades claras, sabemos exactamente qué va a afectar un cambio.

6.2. Patrones de Diseño: ¿Valieron la Pena?

Repository Pattern: Definitivamente sí. Centralizar el acceso a datos nos permitió:

- Optimizar consultas en un solo lugar
- Cambiar la base de datos si fuera necesario (solo afecta repositorios)
- Mockear datos para testing
- Tener control total sobre las operaciones con BD

DTO Pattern (Serializers): Fundamental. Nos protegió de exponer datos sensibles y nos dio validación automática. Además, la documentación de Swagger se genera automáticamente.

Service Layer: Fue el cambio que más impacto tuvo. Separar la lógica de negocio de las vistas hizo que:

- Las vistas sean súper simples (solo reciben y responden)
- La lógica se pueda reutilizar
- Los tests sean más fáciles
- El código sea más legible

6.3. Qué Mejoró

1. **Claridad del código:** Cualquier desarrollador puede entender la estructura en minutos.
2. **Rendimiento:** Las consultas optimizadas redujeron los tiempos de respuesta significativamente.
3. **Seguridad:** Al usar Serializers, controlamos exactamente qué datos se exponen y cuáles no.
4. **Documentación:** Swagger genera documentación automática de todos los endpoints.
5. **Mantenibilidad:** Hacer cambios ya no da miedo.

6.4. Qué Se Podría Optimizar

Aunque mejoramos mucho, todavía hay áreas que se pueden optimizar:

1. **Testing:** Tenemos 45 % de cobertura, pero deberíamos llegar al 70-80 %.
2. **Caché:** Implementar Redis caché para consultas frecuentes podría mejorar aún más el rendimiento.
3. **Optimización de queries:** Algunas consultas todavía pueden optimizarse con `select_related` y `prefetch_related`.
4. **Frontend:** El frontend podría beneficiarse de un estado global (Redux o Context API) en lugar de solo hooks locales.

5. **Logging:** Implementar un sistema de logs más robusto para monitoreo en producción.
6. **CI/CD:** Automatizar completamente el deployment con pruebas automáticas.

6.5. Lecciones Aprendidas

1. **Empezar bien es más barato que refactorizar:** Refactorizar código existente tomó el doble de tiempo que si hubiéramos empezado bien desde el inicio.
2. **Los patrones no son complicados:** Al principio parecían complicados, pero una vez que los entiendes, son bastante lógicos y naturales.
3. **La documentación es clave:** Documentar mientras desarrollas es mucho más fácil que documentar después.
4. **El código limpio se lee como prosa:** Como dice Martin [?], el buen código debería ser fácil de leer. Si no lo es, probablemente está mal diseñado.
5. **Las buenas prácticas no ralentizan, aceleran:** Al principio pensábamos que nos iban a hacer más lentos, pero en realidad aceleraron el desarrollo.

6.6. Relación con Trabajos Previos

Nuestros resultados confirman lo que mencionan Piñero González et al. [?] sobre la eficiencia del desempeño: aplicar buenas prácticas desde el inicio mejora significativamente los tiempos de respuesta y el uso de recursos.

También coincidimos con Mercado & Zapata [?] en que la gestión de calidad con metodologías ágiles facilita el desarrollo y reduce errores. La combinación de buenas prácticas + arquitectura limpia + revisión de código fue clave para nuestro éxito.

Finalmente, como sugieren Espinosa & Eraso [?], la gestión adecuada de tecnología y buenas prácticas no solo mejora el producto, sino también la productividad del equipo y la satisfacción al trabajar.

7. Conclusiones

7.1. Resumen de Hallazgos

Este proyecto demostró que **aplicar patrones de diseño y buenas prácticas no es opcional si quieres software de calidad**. Los principales hallazgos fueron:

1. **La arquitectura en capas funciona:** Separar responsabilidades en Model, Serializer, Repository, Service y ViewSet hace el código más mantenible, testeable y escalable.
2. **Los patrones de diseño son herramientas, no dogmas:** Repository, DTO y Service Layer nos ayudaron específicamente con nuestros problemas. No aplicamos patrones por aplicarlos, sino porque resolvían problemas reales.
3. **El refactoring es doloroso pero necesario:** Transformar código caótico en código limpio tomó tiempo y esfuerzo, pero el resultado valió totalmente la pena.
4. **La calidad se mide en tiempo de desarrollo:** Un sistema bien diseñado permite agregar funcionalidades en horas en lugar de días.

5. **Los números no mienten:** Reducción del 40-60 % en tiempos de respuesta, 67 % menos líneas por función, y 77 % menos código duplicado son resultados concretos y medibles.

7.2. Contribuciones

Este trabajo contribuye con:

1. **Un caso de estudio real:** Mostramos cómo transformar un proyecto caótico en uno estructurado, con ejemplos concretos y código real del sistema de autogestión SENA.
2. **Implementación práctica de patrones:** No solo teoría, sino implementación real de Repository, DTO y Service Layer en un proyecto de producción.
3. **Métricas concretas:** Datos reales de rendimiento antes y después de aplicar buenas prácticas.
4. **Guía para principiantes:** Este artículo puede servir como referencia para otros estudiantes o desarrolladores junior que enfrentan problemas similares.
5. **Demostración del valor de las buenas prácticas:** Prueba tangible de que las buenas prácticas no son “pérdida de tiempo”, sino una inversión que se paga sola.

7.3. Limitaciones

Reconocemos algunas limitaciones del trabajo:

1. **Contexto específico:** Este es un proyecto educativo para el SENA, no un sistema con millones de usuarios. Los resultados pueden variar en contextos diferentes.
2. **Cobertura de testing:** Aunque mejoramos, el 45 % de cobertura es insuficiente para un sistema crítico en producción.
3. **Equipo pequeño:** Con un equipo más grande, quizás hubieran surgido otros problemas o soluciones diferentes.
4. **Tiempo limitado:** Algunos aspectos como CI/CD completo o monitoreo avanzado quedaron pendientes.
5. **Experiencia del equipo:** Como desarrolladores en formación, probablemente hay mejoras que no identificamos por falta de experiencia.

7.4. Trabajos Futuros

Para seguir mejorando, proponemos:

1. **Incrementar cobertura de tests:** Llegar al 80 % con tests unitarios e integración.
2. **Implementar CI/CD completo:** Pipeline automático con tests, linting, y deployment.
3. **Optimización avanzada:** Implementar caché con Redis, indexación en BD, lazy loading en frontend.
4. **Monitoreo en producción:** Herramientas como Sentry para tracking de errores y métricas de performance.
5. **Documentación completa:** Documentar arquitectura, decisiones de diseño y guías de contribución.
6. **Microservicios:** Evaluar si sería beneficioso dividir el monolito en microservicios.
7. **GraphQL:** Considerar GraphQL en lugar de REST para consultas más flexibles.
8. **Progressive Web App:** Hacer el frontend una PWA para funcionalidad offline.

7.5. Reflexión Final

Este proyecto nos enseñó que **el desarrollo de software no es solo hacer que funcione, sino hacerlo bien**. Como dijo Martin [?]: “No basta con que el código funcione; debe ser limpio”.

Al principio, pensábamos que las buenas prácticas y los patrones de diseño eran cosas que solo importaban en empresas grandes o proyectos complejos. Estábamos equivocados. Desde el día uno, aplicar buenas prácticas hace la diferencia.

Si tuviéramos que dar un consejo a otros desarrolladores que están empezando, sería: **No esperes a que el proyecto se vuelva inmantenible para pensar en arquitectura**. Empieza bien desde el principio, aprende patrones de diseño, sigue principios SOLID, escribe código limpio. Te va a ahorrar semanas (o meses) de dolores de cabeza.

Finalmente, este proyecto nos demostró que es posible desarrollar software de calidad siguiendo buenas prácticas, y que el esfuerzo invertido en hacerlo bien se recupera con creces en mantenibilidad, escalabilidad y satisfacción al trabajar.

A. Checklist de reproducibilidad (plantilla)

- **Datos:** fuente, versión, licencias, anonimización.
- **Código:** repositorio, commit hash, instrucciones de ejecución.
- **Entorno:** SO, versión de compiladores, dependencias, semillas.
- **Procedimiento:** pasos exactos para replicar resultados.
- **Resultados:** tablas/figuras generadas automáticamente en build/.

Referencias

- R. D. C. Espinosa and J. C. C. Eraso. 2024. Gestión de tecnología y buenas prácticas empresa de desarrollo de software colombiana. *REVISTA DELOS* 17, 62 (2024), e3210. doi:10.55905/rdelosv17.n62-117
- Robert C. Martin. 2008. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Victor Mercado and Julián Zapata. 2019. Revisión de herramientas y buenas prácticas implementadas para la gestión de la calidad en proyectos de desarrollo de software con metodologías ágiles. (2019). *Especialización en Ingeniería de Software*.
- Maidelyn Piñero González, Aymara Marin Díaz, Yaimí Trujillo Casañola, and Denys Buedo Hidalgo. 2021. Buenas prácticas para prevenir los riesgos de la eficiencia del desempeño en los productos de software. *Revista Cubana de Ciencias Informáticas* 15, 1 (2021).